

LINUX INTERGALÁCTICO: BASE DA
JORNADA JEDI

*TERMINAL DE
COMANDOS
CONQUISTADO!*



5

LISTA DE FIGURAS

Figura 1 – Exemplo de um arquivo .bashrc em um Kali Linux WSL.....	9
Figura 2 – Conteúdo do /etc/profile.d em um Kali Linux WSL	10
Figura 3 – Exemplo do comando ping em <i>background</i> em um Kali Linux WSL	11
Figura 4 – Uso do kill para “matar” um processo em um Kali Linux WSL	13

EMANIP

LISTA DE QUADROS

Quadro 1 – Tabela de padrões relacionais	23
Quadro 2 – Caracteres especiais utilizados pelo awk	26

EMENDAS

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Script para resolução do desafio.....	16
Código-fonte 2 – Resolução utilizando um laço for	18
Código-fonte 3 – Conteúdo do arquivo clientes.txt.....	21

EMSE

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Comando compgen.....	8
Comando de prompt 2 – Comando sed	17
Comando de prompt 3 – Comando grep	18
Comando de prompt 4 – Exemplo de uso do awk.....	19
Comando de prompt 5 – Sintaxe do comando awk.....	20
Comando de prompt 6 – Sintaxe do comando awk com o parâmetro “-f”	20
Comando de prompt 7 – Primeiro exemplo de awk para listagem de dados	21
Comando de prompt 8 – Segundo exemplo de awk para listagem de dados	22
Comando de prompt 9 – Terceiro exemplo de awk para listagem de dados.....	22
Comando de prompt 10 – Quarto exemplo de awk para listagem de dados.....	24
Comando de prompt 11 – Quinto exemplo de awk para listagem de dados	24
Comando de prompt 12 – Sexto exemplo de awk para listagem de dados.....	24
Comando de prompt 13 – Sétimo exemplo de awk para listagem de dados.....	25
Comando de prompt 14 – Oitavo exemplo de awk para listagem de dados	25
Comando de prompt 15 – Nono exemplo de awk para listagem de dados	26

SUMÁRIO

TERMINAL DE COMANDOS CONQUISTADO!.....	7
1 INTRODUÇÃO	7
2 TRABALHANDO COM AS VARIÁVEIS E ALIASES	7
2.1 Variáveis e Aliases	7
3 ARRUMANDO A BAGUNÇA.....	8
4 <i>WHEN THE MAGIC HAPPENS</i>	9
5 CONTROLE DE PROCESSOS.....	11
5.1 Executando comandos.....	11
6 VAMOS DESAFIAR NOSSO CONHECIMENTO?	14
7 OLÁ, MEU NOME É AWK.....	19
8 LISTANDO DADOS.....	21
9 AFINAL DE CONTAS, TUDO SÃO PADRÕES.....	23
10 COMO FICAM AS EXPRESSÕES REGULARES?.....	24
11 E ONDE ENTRAM O BEGIN E O END?	26
CONCLUSÃO.....	27
REFERÊNCIAS.....	28

TERMINAL DE COMANDOS CONQUISTADO!

1 INTRODUÇÃO

Continuando nosso desbravamento no ambiente Linux, neste capítulo, iremos subir mais um degrau para entender como funciona o terminal (Bash), como podemos customizá-lo e quais as suas possibilidades.

De acordo com Silva (2018), os dois principais métodos para personalizar o bash são as opções e as variáveis de ambiente. Algumas opções são definidas usando o comando `set`. Outras opções são estabelecidas usando o comando `shopt` (*shell options*). As variáveis de ambiente são definidas cada vez que o Bash inicia a execução. Você pode alterar opções e variáveis a qualquer momento, mas não precisa configurar o Bash toda vez que abrir o terminal, pode defini-lo nos arquivos `.bash_profile` e `.bashrc`. Para explicar como esses arquivos funcionam, primeiro, devemos determinar as variáveis do Bash.

2 TRABALHANDO COM AS VARIÁVEIS E ALIASES

2.1 Variáveis e Aliases

Conforme Silva (2018), as variáveis são usadas para armazenar qualquer tipo de valor, incluindo valores de retorno do programa (por exemplo, **variável=\$(programa_em_execução)**, em que a variável receberá a saída do comando como entrada).

No Linux, definimos variáveis assim: **nome=valor**, sem espaços antes ou após o "=", se o valor da variável contiver um ou mais espaços, este deve estar entre aspas. E para obter o valor de uma variável, basta executar **echo \$nome**, sendo o \$ seguido do nome da variável responsável por trazer seu valor à tela.

Da mesma forma, você pode definir aliases para encurtar comandos comuns difíceis de digitar. Por exemplo, você pode definir que o comando **ls -lah** será executado quando você digitar **la** usando o seguinte comando: **alias la="ls -lah"**.

3 ARRUMANDO A BAGUNÇA

Conforme Silva (2018), o Bash possui basicamente dois arquivos para personalizar o ambiente. O primeiro arquivo é o **.profile**, localizado no diretório inicial de cada usuário (home). Esse arquivo geralmente é usado para definir variáveis e opções de ambiente, como configurações-padrão e seleção do método de trabalho do Bash. A lista de variáveis do usuário pode ser obtida assim:

```
compgen -v | while read line ; do echo \ $line =\${! line };  
done
```

Comando de prompt 1 – Comando compgen
Fonte: Elaborado pelo autor (2019)

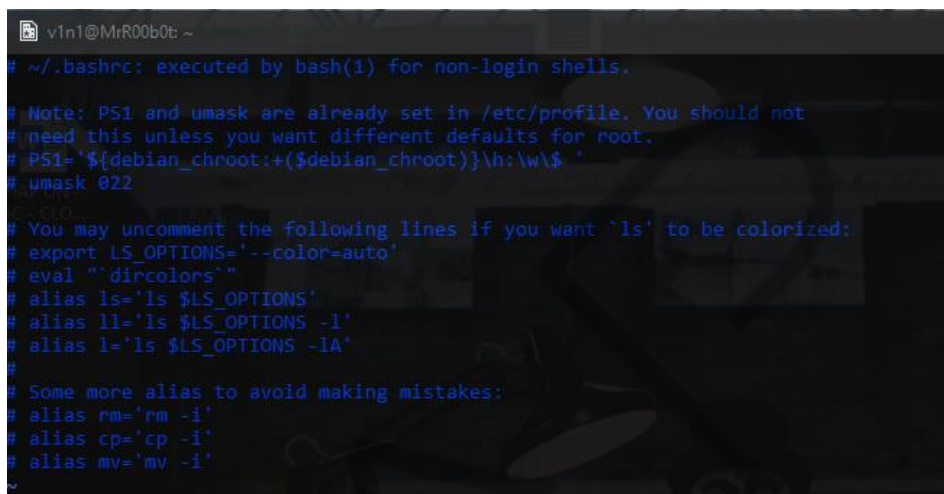
No comando de prompt “Comando compgen”, o comando while é usado apenas para imprimir o conteúdo de variáveis que contêm várias linhas.

Existem diversas opções e variáveis de ambiente, por isto, será mais eficiente encontrá-las com base nas necessidades de determinadas configurações específicas. Esse não é o caso da variável **\$PATH**, pois alguns aplicativos exigem que o usuário altere seu conteúdo, que contém o caminho do executável.

Portanto, se você deseja tornar o executável acessível sem precisar fazer referência ao caminho do executável, a recomendação é: esses executáveis criam diretórios e adicionam o caminho do diretório à variável **\$PATH**. Para adicionar um caminho à variável PATH sem substituir seu conteúdo, atribua **PATH=\$PATH“:/home/path/from/directory”**.

O outro arquivo que comentamos sobre personalização do Bash é o **.bashrc**, que também se encontra na home do usuário. O arquivo geralmente contém configurações de preferências pessoais, como alterar dicas ou cores fora da definição de alias. O **.bashrc** é um arquivo de controle de execução usado por muitos aplicativos como um arquivo de instrução e comando associado a um programa que o interpreta na inicialização do sistema.

O conteúdo desses dois arquivos é apenas uma boa prática para facilitar a identificação dessas configurações, e o ideal é que as alterações feitas sejam comentadas. Como mencionado anteriormente, este material não descreve todas as opções de configuração, já que há tantas opções que os leitores dificilmente se lembram de todos os detalhes necessários.



```
vini@Mr00b0t: ~  
# ~/.bashrc: executed by bash(1) for non-login shells.  
  
# Note: PS1 and umask are already set in /etc/profile. You should not  
# need this unless you want different defaults for root.  
# PS1='${debian_chroot:+($debian_chroot)}\h:\w\$ '  
# umask 022  
#  
# You may uncomment the following lines if you want 'ls' to be colored:  
# export LS_OPTIONS='--color=auto'  
# eval "$(dircolors)"  
# alias ls='ls $LS_OPTIONS'  
# alias ll='ls $LS_OPTIONS -l'  
# alias l='ls $LS_OPTIONS -la'  
#  
# Some more alias to avoid making mistakes:  
# alias rm='rm -i'  
# alias cp='cp -i'  
# alias mv='mv -i'
```

Figura 1 – Exemplo de um arquivo .bashrc em um Kali Linux WSL
Fonte: Elaborado pelo autor (2019)

Há também o arquivo **.bash_logout**, que é responsável pelas execuções de scripts quando do encerramento do bash.

4 WHEN THE MAGIC HAPPENS

Quando acessamos a área gráfica de um sistema Linux qualquer, por meio dela, abrimos um terminal já “logado”, ou seja, se o usuário já está manipulando o “shell gráfico” desse ambiente (usuário corrente), ao abrir um terminal, o login deste novo shell é executado automaticamente. Nesse exemplo que acabamos de citar ou em qualquer outro exemplo de abertura de um terminal em que haja um login, inclusive uma conexão ssh, esse novo shell é chamado de “**login shell**”.

Conforme explicado por Silva (2018), após o logon do shell, é executado o script **/etc/profile**, que inicializa as variáveis necessárias e executa os scripts presentes no diretório **profile.d**. Quaisquer alterações no processo devem ser incluídas como scripts nesse diretório, para que as alterações no **/etc/profile** não sejam substituídas quando o sistema for atualizado.

Finalmente, o perfil do usuário será executado e pode ter um nome diferente devido à compatibilidade com versões anteriores do Bash. A ordem de prioridade entre os perfis de usuário é da maior para a menor:

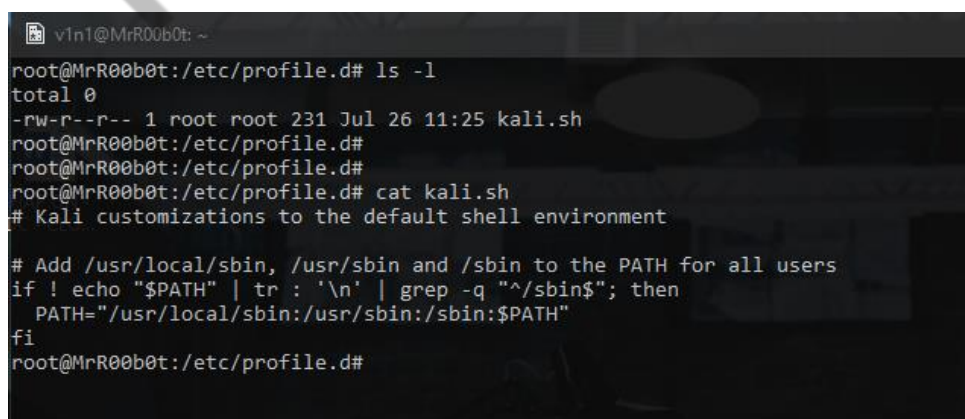
- **.bash_profile**
- **.bash_login**
- **.profile**

Sabendo que somente os arquivos de maior prioridade são executados, no final desse processo, o arquivo **.bashrc** é executado, também contendo definições específicas do usuário. Um dado interessante é que, em alguns sistemas, o **.bashrc** compartilhado também é executado para todos os usuários.

Para iniciar um shell interativo, o processo é mais simples: mesmo que essa execução não seja especificada no **.profile**, o arquivo **bashrc** (arquivo para todos os usuários e para o usuário específico que abriu o bash) será executado porque a execução do **.bashrc** é realizada diretamente pelo shell interativo.

Não é novidade que o Bash não possui um menu gráfico com botões e opções de configuração, mas os usuários podem configurá-lo com variáveis e opções predefinidas (set e shopt). Portanto, não é necessário configurar o Bash toda vez que você o inicia, porque ele possui os arquivos de configuração que estamos apresentando. Ao executá-los, suas alterações não serão refletidas no shell lançado antes delas.

O uso de arquivos de controle de execução não é exclusivo do Bash e pode ser encontrado em editores de texto, como o Vim.



```
v1n1@MrR00b0t: ~  
root@MrR00b0t:/etc/profile.d# ls -l  
total 0  
-rw-r--r-- 1 root root 231 Jul 26 11:25 kali.sh  
root@MrR00b0t:/etc/profile.d#  
root@MrR00b0t:/etc/profile.d#  
root@MrR00b0t:/etc/profile.d# cat kali.sh  
# Kali customizations to the default shell environment  
  
# Add /usr/local/sbin, /usr/sbin and /sbin to the PATH for all users  
if ! echo "$PATH" | tr : '\n' | grep -q "^/sbin$"; then  
    PATH="/usr/local/sbin:/usr/sbin:/sbin:$PATH"  
fi  
root@MrR00b0t:/etc/profile.d#
```

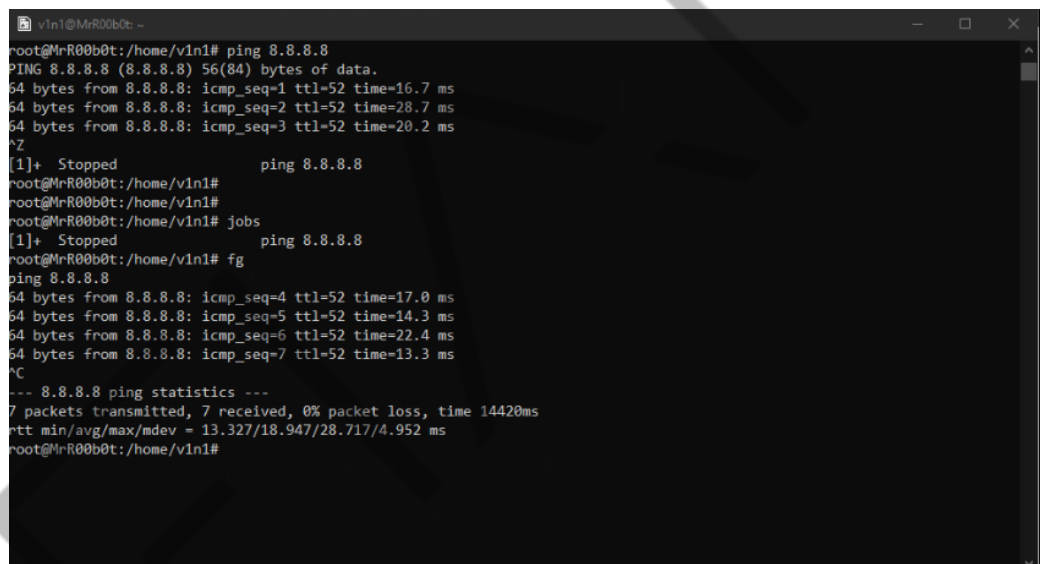
Figura 2 – Conteúdo do /etc/profile.d em um Kali Linux WSL

Fonte: Elaborado pelo autor (2019)

5 CONTROLE DE PROCESSOS

5.1 Executando comandos

De acordo com Silva (2018), ao executar algum comando no Bash, um processo é iniciado em primeiro plano (*foreground*), interrompendo a execução do bash e podendo ler a entrada do teclado. É possível iniciar um processo (o qual chamamos de **job**) no background pelo Bash, inserindo o caractere “&” ao final da linha de comando (Por exemplo: **ping -c4 8.8.8.8 &**, em que executaríamos o envio de quatro pacotes ICMP, colocando a execução do referido comando em *background*).



```
root@MrR00b0t:/home/vin1# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=52 time=16.7 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=52 time=28.7 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=52 time=20.2 ms
^Z
[1]+  Stopped                  ping 8.8.8.8
root@MrR00b0t:/home/vin1#
root@MrR00b0t:/home/vin1#
root@MrR00b0t:/home/vin1# jobs
[1]+  Stopped                  ping 8.8.8.8
root@MrR00b0t:/home/vin1# fg
ping 8.8.8.8
64 bytes from 8.8.8.8: icmp_seq=4 ttl=52 time=17.0 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=52 time=14.3 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=52 time=22.4 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=52 time=13.3 ms
^C
--- 8.8.8.8 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 14420ms
rtt min/avg/max/mdev = 13.327/18.947/28.717/4.952 ms
root@MrR00b0t:/home/vin1#
```

Figura 3 – Exemplo do comando ping em *background* em um Kali Linux WSL
Fonte: Elaborado pelo autor (2019)

Quando o Bash cria um trabalho em segundo plano (o qual chamamos de **job**), ele imprime uma linha contendo o número **job** e o ID do processo do último pipeline. Observe, no entanto, que, em alguns casos (como no exemplo acima), para liberar o terminal novamente para o usuário, você deve pressionar “Enter” para finalizar a execução em segundo plano.

Para simplificar o gerenciamento de processos, o Bash rastreia o trabalho atual (o último trabalho parado em primeiro plano ou o último processo iniciado em segundo plano) e o anterior.

Como apresentado por Silva (2018), o comando "**ctrl + z**", na maioria dos terminais, interrompe o processo em primeiro plano, liberando o prompt para o usuário. O comando **fg** é usado para executar tarefas em primeiro plano e o comando **bg** é usado para executar tarefas em segundo plano. O comando **jobs** é responsável por listar os trabalhos ativos.

Quando o trabalho em segundo plano é concluído, o Bash imprime a linha que contém o número de série e o comando que o invocou (para não interromper a execução em primeiro plano, essa impressão aguardará até que o Bash execute a impressão novamente).

Quando nenhum parâmetro é passado, os comandos "**fg**" e "**bg**" assumem o trabalho atual, mas você pode usá-los em outros trabalhos com uma especificação de trabalho **jobspec**, que é uma referência ao **job** e começa com o caractere "%", seguido pelo número de série do trabalho ou pelo comando de chamada do trabalho. Ainda é possível utilizar o curinga **?** como uma *substring* do comando de chamada de tarefa. Assim sendo, o caractere **%+** é usado para referenciar o trabalho atual e o caractere **%-** é usado para referenciar o trabalho anterior, por exemplo:

- fg %5
- bg % vim
- fg %? vi
- fg %-

Você também pode enviar sinais para trabalhos usando o comando **kill**. O comando **kill** aceita o tipo de sinal enviado como uma opção e **jobspec** ou **id** do processo como parâmetros. A lista completa de sinais pode ser obtida usando o comando **kill -l**. O sinal mais comum é o **SIGKILL** (9), que força o final da execução.

Ainda conforme Silva (2018), o comando **kill** permite que um sinal seja enviado a todos os processos em execução no computador por intermédio do ID do processo, incluindo os trabalhos de shell que não estejam em uso. O comando **ps aux** é uma maneira de obter um processo em execução.

O comando **ps** retorna uma lista de todos os processos em execução e o **aux** são três parâmetros usados para exibir os processos de todos os usuários, o nome de usuário responsável pelo processo e os processos não executados pelo Bash. Você pode usar o comando **ps** com o parâmetro **f** para exibir a árvore de processos ou o comando **pstree** para imprimir a árvore de forma mais legível. Isso é útil para determinar quais processos serão finalizados.

Usando o comando **top** ou **htop**, você será capaz de exibir a lista de processos e seu ID e outras informações com mais detalhes. O **htop** exibe ainda uma lista interativa que permite aos usuários filtrar, pesquisar, classificar e até enviar sinais para os processos listados.

Uma ferramenta útil no dia a dia é o comando **fuser**, que mostra o processo de determinados arquivos ou diretórios transmitidos usando parâmetros. Dessa forma, quando um usuário é impedido de excluir ou mover um arquivo, ele pode encontrar um processo que pode causar um obstáculo e encerrá-lo.

Você também pode iniciar um processo, que pode ser executado mesmo que o usuário responsável efetue logout, utilizando o comando **nohup**. Por meio desse comando (e usando redirecionamentos de entrada e saída), você pode executar operações longas que facilitemo uso do computador para outros usuários.

O controle do processo é essencial para os administradores de sistema, porém, ainda é muito útil para os usuários comuns.



```
v1n1@MrR00b0t: ~
root@MrR00b0t:~# ping 8.8.8.8 2>&1 > /dev/null &
[1] 30
root@MrR00b0t:~# ps aux | grep ping
root      30  0.1  0.0 14856 1092 tty1      S   09:30   0:00 ping 8.8.8.8
root      32  0.0  0.0 14052 1136 tty1      S   09:31   0:00 grep ping
root@MrR00b0t:~# kill -9 30
root@MrR00b0t:~#
[1]+  Killed                  ping 8.8.8.8 2>&1 > /dev/null
root@MrR00b0t:~#
```

Figura 4 – Uso do kill para “matar” um processo em um Kali Linux WSL
Fonte: Elaborado pelo autor (2019)

Na imagem da Figura “Uso do kill para “matar” um processo em um Kali Linux”, executamos o **ping** para o endereço **8.8.8.8**, enviando a saída padrão (stdout, **2**) para saída de erro (stderr, **1**) e ambas para o “buraco negro do Linux” (**/dev/null**), colocando esse comando em background (**&**).

Depois disso, efetuamos uma busca pelo comando ping nos processos em execução (**ps aux | grep ping**) e enviamos o sinal SIGKILL (9) para que fosse forçado a encerrar (também chamado “matar o processo”). Dessa forma, podemos ver que o Shell nos informou que o processo foi encerrado por meio da informação **[1]+ killed** exibida no terminal.

6 VAMOS DESAFIAR NOSSO CONHECIMENTO?

O exercício a seguir, proposto por Silva (2018), levará você a uma solução automatizada que usa apenas o Bash (e um auxiliar chamado pelo Bash) para filtrar e separar dados específicos de um arquivo. A solução inclui procurar ferramentas que possam ser úteis para a solução, como **sed**, **awk**, **cut** etc. (para aproveitar ao máximo o Bash, você precisa conhecer ou poder procurar ferramentas que ajudem a resolver o problema).

Teremos como base o arquivo infra.csv:

...

" 447676 ";" SWITCH ";" RACK DELL ";" ";" 2100.17.07.31 ";

" 447677 ";" SWITCH ";" RACK DELL ";" ";" 2100.17.07.29 ";

" 447678 ";" SWITCH ";" RACK DELL ";" ";" 2100.17.07.30 ";

" 447679 ";" SWITCH ";" RACK DELL ";" ";" 2100.17.07.28 ";

" 452782 ";" SERVIDOR BLADE ";" BASE A GAS ";" FUNPAR ";" 2100.17.07 ";

" 460805 ";" FORNO DE MICRO-ONDAS ";" 30L ";" CONSUL ";" 2100.17.02.24 ";

" 460806 ";" SERVIDOR TORRE (5234) ";" 127V ";" ";" 2100.17.02.21 ";

" 460807 ";" SERVIDOR TORRE (5234) ";" 127V ";" ";" 2100.17.02.21 ";

```
" 460808 ";" SERVER TORRE (5234) ";"127V ";""";" 2100.17.02.21 ";
```

Em cada linha, o conteúdo das colunas está entre aspas (" "), separado por ponto e vírgula (;). Porém, o conteúdo de alguns campos pode conter o caractere (;).
Exemplo:

Original:

```
" 460808 SERVER TORRE ; CINZA ";"127V ";""";" 2100.17.02.21 ";
```

Filtrado:

```
" 460808 ";" SERVER TORRE CINZA ";"127V ";""";" 2100.17.02.21 ";
```

A primeira etapa do exercício é filtrar todo o conteúdo do arquivo e remover essas ocorrências. A segunda etapa é, a partir do resultado filtrado, obter, em um arquivo, todos os locais (conteúdo da quinta coluna), um em cada linha, ordenados e sem repetição. Exemplo:

...

2100.17.02.21

2100.17.02.23

2100.17.02.24

2100.17.02.25

2100.17.02.29

2100.17.07.01

2100.17.07.02

2100.17.07.03

2100.17.07.04

...

Por fim, deve-se criar um arquivo para cada local com o nome sendo o código do local com a extensão “.csv”. O conteúdo de cada arquivo deve ser as linhas do arquivo original em que o campo de local corresponde ao nome do arquivo. Exemplo: 2100.17.csv:

...

```
" 480170 ";"NO - BREAK (5230) ";"";"";" 2100.13 ";
" 480171 ";" MICROCOMPUTADOR ";"";"";" 2100.13 ";
" 480172 ";" NOTEBOOK ";"";"";" 2100.13 ";
" 480173 ";" MONITOR DE VIDEO (5235) ";"";"";" 2100.13 ";
" 480174 ";" NOTEBOOK ";"";"";" 2100.13 ";
" 480175 ";" MICROCOMPUTADOR ";"";"";" 2100.13 ";
```

...

Para tal, Silva (2018) propôs o seguinte script:

```
#!/ bin/sh
# remover ";" e gerar locais .txt
cat infra .csv | sed -E 's /(([^ "] ) (;) | (;) ([^"] ))
*/\2/g' | grep
-oE ' " ([0 -9]+(\. [0 -9]+) +) ";" ' | grep -oE ' [0 -
9]+(\. [0 -9]+) +' |
sort -u > locais .txt;
# se não existir , cria diretório locais
mkdir -p locais
# para cada local , preencher os dados
for i in $( cat locais .txt); do
cat infra .csv | grep "\"$i\"";" >> locais /$i.csv;
done
```

Código-fonte 1 – Script para resolução do desafio
Fonte: SILVA (2018)

Vamos dissecar o script do Código-Fonte “Script para resolução do desafio”.

De acordo com Silva (2018), a maneira mais intuitiva de resolver o problema é um editor de fluxo de texto (ele não acessa arquivos por local, mas linearmente, por meio de um buffer). Portanto, basta usar expressões regulares para reconhecer padrões e eliminar caracteres indesejados (;). Ao procurar soluções semelhantes em sites de perguntas e respostas e em tutoriais, você geralmente encontrará soluções para eliminar tudo no delimitador, não apenas eliminar caracteres específicos no delimitador.

O **sed** tem uma solução necessária, mas pode ser difícil de encontrar, porque a maioria dos tutoriais é muito antiga e o uso do **sed** pode variar entre diferentes tipos de shell. As possíveis soluções para a sintaxe do **sed** são:

```
sed -E 's /([^[ "]) (;) | (;) ([^"])) */\2/g'
```

Comando de prompt 2 – Comando sed
Fonte: Silva (2018)

A opção **-E** permite expressões regulares (ER) estendidas, necessárias para a exclusão **sed**, entretanto, mantém o resto. A ER procurará por “;” não sendo seguidos imediatamente por “ ”.

Desse modo, o sinal de ponto e vírgula separado fica entre "ambos os lados" e a expressão não é detectada, é apenas retida; deve ser isolada.

A opção **s///** substitui o padrão encontrado de outras maneiras, **\2** é o segundo padrão identificado (os padrões são identificados por "(" e ")") na ordem associada. Portanto, o padrão correspondente será substituído pelo segundo elemento encontrado (No primeiro caso da OR, não na parte ";" e, no segundo caso, ele não faz nada porque corresponde ao primeiro. Parte do padrão também não corresponde.).

Essa solução não é perfeita se for “;” o primeiro caractere, porque foi encontrada no segundo caso ou, nesse evento, a parte do padrão a ser retida não é mais a segunda encontrada. Padrões. Para corrigir isso, você pode usar outra expressão regular ou executar dois seds e, em seguida, executar um para cada caso.

No retorno do **sed** anterior, você pode usar o **grep** para selecionar apenas os locais. A opção **-E** permite o uso de expressões regulares e **-o** para retornar apenas o padrão encontrado, não as linhas que contêm o padrão. No primeiro **grep**, apenas os campos cujo conteúdo completo é somente o código local são selecionados e, em seguida, o próximo **grep** pesquisa apenas o código, sem o " e ; para isolar o conteúdo. Dessa maneira, mesmo que outro campo cite um código de localização, ele não será detectado pelo **grep**, porque o conteúdo da célula não é apenas o código de localização.

Outra maneira de isolar apenas a coluna de lugar é usar um editor de fluxo para separar os campos (depois de resolver o problema; no conteúdo). Contudo, se uma coluna for adicionada antes disso, o script deverá ser alterado para selecionar a coluna correta. O ponto negativo da primeira solução é que, se for criada uma outra coluna de referência a locais, duas colunas serão selecionadas. Independentemente da maneira escolhida, com os códigos de locais isolados, basta ordenar, removendo repetições, e redirecionar a saída.

```
grep -oE ' " ([0 -9]+(\.[0 -9]+) +)";' | grep -oE '[0 -9]+(\.[0 -9]+) +' | sort -u > locais .txt;
```

Comando de prompt 3 – Comando grep
Fonte: Silva (2018)

Finalmente, para cada linha do arquivo locations.txt (ou seja, para cada local), as linhas do arquivo original que contêm esse local são selecionadas. O ">>" redireciona a concatenação do que já foi gravado, para que cada linha seja adicionada sem substituir o conteúdo anterior.

```
for i in $( cat locais .txt); do  
  cat patrimonio .csv | grep "\"$i\""; >> locais  
/$i.csv;  
done
```

Código-fonte 2 – Resolução utilizando um laço for
Fonte: Silva (2018)

Existem várias soluções possíveis, usando várias ferramentas e diferentes maneiras de encontrar partes do texto. A solução deve levar em consideração o contexto dos dados, por exemplo, se é comum alterar a estrutura do arquivo com mais colunas, uma solução fácil de alterar ou até obter nomes de arquivos, caracteres separadores ou números de colunas, conforme os parâmetros.

7 OLÁ, MEU NOME É AWK

Uma das atividades mais corriqueiras na vida de um profissional de Defesa Cibernética é o tratamento de arquivos de texto. Esta atividade pode ser desempenhada por diversos motivos: seja em uma análise de forense em que precise concatenar dados de diversos arquivos, seja num tratamento de incidentes, em que a leitura e a correlação dos logs se fazem imprescindíveis, entre outras.

Entretanto, realizar essa atividade com softwares gráficos, dependendo do tamanho da entrada de dados que necessita de processamento, seria um tanto quanto desastroso. Para isso, neste capítulo, iremos apresentar o AWK, um poderoso “faz tudo” em ambientes Linux. Uma excelente ferramenta para resolver desde os problemas simples até os mais complexos que aparecem no dia a dia de usuários e administradores.

De acordo com Vieira (2010), a função do **awk** é analisar um conjunto de linhas de entrada uma a uma, procurando aquelas que atendem a determinados padrões ou condições especificadas pelo usuário. Veja a seguinte estrutura básica:

[padrão / condição] [(ação)]

Exemplo:

```
awk '$1 == "Vinicius" { print $2, $3 }' arquivo1
```

Comando de prompt 4 – Exemplo de uso do awk
Fonte: Elaborado pelo autor (2019)

O exemplo acima ilustra uma instrução **awk** típica, ou seja, o tipo de padrão de ação. A instrução imprime o segundo e o terceiro campos de cada linha do arquivo file1 quando o primeiro campo é igual a "Vinicius". Em resumo, o comando **awk** executará a ação especificada (que pode executar várias etapas) para cada padrão ou condição encontrada na linha que você está lendo.

Ainda de acordo com Vieira (2010), existem duas formas de executar o **awk**:

Pela linha de comando:

```
awk '<padrao-acao>' [arquivo_1] [arquivo_2] ... [arquivo_n]
```

Comando de prompt 5 – Sintaxe do comando awk
Fonte: Elaborado pelo autor (2019)

Observe que o padrão e a ação aparecem nos apóstrofes. Isso é para impedir que o Shell interprete caracteres especiais, como o cifrão (\$), e também para permitir que o comando estenda quantas linhas forem necessárias.

Por meio de um arquivo de comando (script):

```
awk -f <arquivo de programa> <lista de arquivos de entrada>
```

Comando de prompt 6 – Sintaxe do comando awk com o parâmetro "-f"
Fonte: Elaborado pelo autor (2019)

Os dados são lidos linha por linha, que, por padrão, formam uma sequência terminada por uma nova linha (ASCII 10). E cada linha (ou registro) lida é dividida em campos, que, por padrão, são uma sequência separada por <TAB> (ASCII 9) ou por espaços em branco.

Com o intuito de facilitar a prática dos assuntos abordados, abra seu editor de textos preferido e crie o arquivo **professores.txt** com o seguinte conteúdo:

Humberto Delgado	Salvador	555-7777
Henrique Poyatos	Campinas	222-4444
Rafael Silva	Manaus	333-5555
Vinicius Vieira	Curitiba	777-4444
Cleiton Alves	Florianópolis	888-9999
Osmany Arruda	Salvador	666-3333

Código-fonte 3 – Conteúdo do arquivo clientes.txt

Fonte: Elaborado pelo autor (2019)

Conforme mostrado por Vieira (2010), cada registro possui os campos: nome da pessoa, cidade e número de telefone separados por <TAB>. E no campo de nome, o primeiro nome é separado do segundo por um espaço em branco. Nesse caso, temos quatro campos, porque, por padrão, o **awk** interpreta qualquer espaço em branco como um separador de campos.

No entanto, podemos definir o separador que queremos usar. Se o definíssemos como apenas <TAB>, por exemplo, os registros teriam apenas três campos. Independentemente, o primeiro campo de um registro será nomeado **\$1**, o segundo campo **\$2** e assim por diante. O conjunto de todos os campos, ou seja, o registro inteiro será chamado de **\$0**.

8 LISTANDO DADOS

Para fazer uma lista simples de um arquivo usando o **awk**, você pode utilizar o seguinte comando:

```
$ awk '{ print }' professores.txt
Humberto Delgado    Salvador    555-7777
Henrique Poyatos    Campinas    222-4444
Rafael Silva        Manaus      333-5555
Vinicius Vieira      Curitiba    777-4444
Cleiton Alves        Florianópolis 888-9999
Osmany Arruda        Salvador    666-3333
```

Comando de prompt 7 – Primeiro exemplo de awk para listagem de dados

Fonte: Elaborado pelo autor (2019)

Repare que a saída é idêntica ao próprio arquivo. E se precisarmos exibir somente o primeiro e o terceiro campos, use **\$1** e **\$3**, desta forma:

```
$ awk '{ print $1, $3 }' professores.txt
Humberto Salvador
Henrique Campinas
Rafael Manaus
Vinicius Curitiba
Cleiton Florianópolis
Osmany Salvador
```

Comando de prompt 8 – Segundo exemplo de awk para listagem de dados
Fonte: Elaborado pelo autor (2019)

Observe que apenas o primeiro nome foi exibido, pois o **awk**, por padrão, interpreta o espaço em branco <TAB> como um separador de campos. Portanto, para o comando, o campo da cidade está na posição 3. Agora, se você quiser que o separador seja <TAB>, digite o comando abaixo:

```
$ awk 'BEGIN{FS="\t"}{print $1, $3}' professores.txt
Humberto Delgado 555-7777
Henrique Poyatos 222-4444
Rafael Silva 333-5555
Vinicius Vieira 777-4444
Cleiton Alves 888-9999
Osmany Arruda 666-3333
```

Comando de prompt 9 – Terceiro exemplo de awk para listagem de dados
Fonte: Elaborado pelo autor (2019)

Perceba que o nome agora está completamente listado e o campo de telefone é considerado o terceiro campo (\$3). Explicaremos mais adiante quais são as opções BEGIN e FS, mas o que podemos antecipar é que a opção FS (Field Separator) foi usada para alterar o separador de campo padrão para <TAB> (\t).

9 AFINAL DE CONTAS, TUDO SÃO PADRÕES

Conforme apresentado por Vieira (2010), o **awk** possui três maneiras de definir padrões pelos quais executa suas ações:

- **Padrões relacionais** – selecionam registros que atendem a certas condições de comparação.
- **Padrões regulares** – selecionam registros que contêm determinadas cadeias.
- **Padrões especiais** – os famosos BEGIN e END que servem para determinar as instruções a serem executadas ANTES de processar o primeiro registro e APÓS o último, respectivamente.

O padrão relacional é usado para comparação e os operadores utilizados são aqueles definidos pelo idioma C, segundo o quadro “Tabela de padrões relacionais”.

Operador	Resultados
==	Igual a
>	Maior que
>=	Maior ou igual
<	Menor que
<=	Menor ou igual

Quadro 1 – Tabela de padrões relacionais
Fonte: Vieira (2010)

Usando os operadores lógicos && (E), || (OU) e ! (NÃO), é possível montar expressões compostas combinando operadores relacionais simples com operadores lógicos.

Imagine que você procure pelo número de telefone de uma pessoa cujo segundo nome é conhecido por começar com a letra V ou H. Você pode usar o comando abaixo:

```
$ awk '$2 >= "V" || $2 >= "H" { print }' professores.txt
Humberto Delgado      Salvador  555-7777
Henrique Poyatos      Campinas 222-4444
Vinicius Vieira       Curitiba 777-4444
```

Comando de prompt 10 – Quarto exemplo de awk para listagem de dados
Fonte: Elaborado pelo autor (2019)

10 COMO FICAM AS EXPRESSÕES REGULARES?

Conforme Vieira (2010), havendo a necessidade de buscar em arquivo por uma string usando expressões regulares, devemos incluir o padrão a ser encontrado em um par de barras (/). Por exemplo, se quisermos listar os registros dos clientes onde os nomes começam com a letra "H", usáramos o seguinte comando:

```
$ awk '/H/ { print }' professores.txt
Humberto Delgado      Salvador  555-7777
Henrique Poyatos      Campinas 222-4444
```

Comando de prompt 11 – Quinto exemplo de awk para listagem de dados
Fonte: Elaborado pelo autor (2019)

No exemplo acima, funcionou a sintaxe da nossa expressão regular. Não existiam outros nomes (incluindo sobrenomes) que começavam com a letra "H". Caso existissem, daria errado. Já na sintaxe abaixo, temos uma expressão regular que buscará somente pelos nomes (primeiros nomes) que comecem com a letra "H":

```
$ awk '$1 ~ /H/ { print }' professores.txt
Humberto Delgado      Salvador  555-7777
Henrique Poyatos      Campinas 222-4444
```

Comando de prompt 12 – Sexto exemplo de awk para listagem de dados
Fonte: Elaborado pelo autor (2019)

Ainda conforme Vieira (2010), e segundo já vimos em Shell Script, os símbolos `() [] \ ^ $ . * ? + |` são um metacaracteres com significados especiais. Os caracteres `^` e `$` são usados, por exemplo, para pesquisar o início e o fim de uma sequência, respectivamente. O conjunto de caracteres entre colchetes (`[]`) procura registros com um desses caracteres. Então, se você deseja listar os registros cujos nomes ou sobrenomes começam com "R" ou "S", faça o seguinte:

```
$ awk '$1 ~ /^[RS]/ || $2 ~ /^[RS]/ { print }' clientes
Rafael Silva      Manaus      333-5555
Vinicius Vieira   Curitiba    777-4444
```

Comando de prompt 13 – Sétimo exemplo de awk para listagem de dados
Fonte: Elaborado pelo autor (2019)

Podemos usar a seguinte estrutura para procurar valores nestes intervalos: padrão1, padrão2, conforme mostrado no exemplo a seguir, em que precisamos de registros cujos nomes de pessoa contenham as letras "O" a "V":

```
$ awk '$1 ~ /O/ , /V/ { print }' professores.txt
Rafael Silva      Manaus      333-5555
Vinicius Vieira   Curitiba    777-4444
Osmany Arruda     Salvador    666-3333
```

Comando de prompt 14 – Oitavo exemplo de awk para listagem de dados
Fonte: Elaborado pelo autor (2019)

Além dos metacaracteres citados anteriormente, o **awk** reconhece os seguintes caracteres, conforme visto no quadro "Caracteres especiais utilizados pelo awk".

Sequência	Significado
\n	Newline
\r	Carriage return
\b	Backspace
\f	FormFeed
\t	<TAB>
\ddd	Valor octal ddd

Quadro 2 – Caracteres especiais utilizados pelo awk
Fonte: Vieira (2010)

11 E ONDE ENTRAM O BEGIN E O END?

Quando precisamos fazer algum processamento antes que o **awk** leia o primeiro registro, usamos o modo BEGIN (por exemplo, execute o cabeçalho). Se o processamento for necessário após a leitura do último registro, o critério END deverá ser usado (por exemplo, gerando um total) (VIEIRA, 2010).

Ou seja, para colocar um cabeçalho e uma mensagem "fim do registro" no rodapé ao ler um arquivo de cliente, você pode usar o seguinte comando:

```
$ awk 'BEGIN { print "Nome\t\t\tCidade\t\t\tTelefone" } { print }
END { print "Fim dos registros" }' professores.txt
Nome Cidade Telefone
Humberto Delgado    Salvador    555-7777
Henrique Poyatos    Campinas   222-4444
Rafael Silva        Manaus     333-5555
Vinicius Vieira     Curitiba   777-4444
Cleiton Alves       Florianópolis 888-9999
Osmany Arruda       Salvador   666-3333
```

Comando de prompt 15 – Nono exemplo de awk para listagem de dados
Fonte: Elaborado pelo autor (2019)

Fim dos registros.

CONCLUSÃO

O **awk** é poderosíssimo e talvez por isso seja considerado por muitos como uma linguagem de programação completa. Porém, a ideia aqui é apresentá-los a algumas das funcionalidades mais usuais no dia a dia da Cyber Security, no que diz respeito ao tratamento de arquivos de texto. Agora, imagine o poder que você teria nas mãos ao aliar o **awk** a outras ferramentas que vimos até agora, como o **grep**, por exemplo? O céu é o limite!

REFERÊNCIAS

SILVA, G. B. B. da. **Tutorial do Bash**. Paraná: Universidade Federal do Paraná, 2018.

VIEIRA, F. D. **Tutorial do comando AWK**. Campinas: Embrapa, 2010.

EMBRAPA